ОТЧЕТ ПО ПРОЕКТУ

по дисциплине «Алгоритмы и структуры данных»

«SORTLAB — интерактивная лаборатория алгоритмов сортировки»

1. Название проекта и состав команды

Полное наименование проекта: «SORTLAB — виртуальная лаборатория для наглядного изучения, пошаговой трассировки и сравнения алгоритмов сортировки».

Краткая концепция: интерактивное веб-приложение (SPA) для визуализации 8 алгоритмов сортировки по принципу «белого ящика» с контролем воспроизведения, сбором метрик и аналитикой.

Таблица 1 — Состав команды разработчиков и распределение ролей

Участник команды (ФИО, группа)	Роль в проекте	Зона ответственности и выполненные работы
Оджа Эльдар	Team Lead / Core Developer (Лидер команды)	Проектирование архитектуры генератора шагов SortingStep, координация работы, разработка ядра визуализации
Сеит-Велиев Эмир	UI/UX & Visualization Engineer (Разработчик)	Реализация 8 алгоритмов сортировки на TypeScript без использования sort(), стратегии выбора Pivot, поддержка сортировки по убыванию
Османов Руслан	Data & Benchmark Engineer (Аналитик / Докладчик)	Расчет метрик Big O, проведение высокоточных бенчмарков performance.now(), сбор данных для аналитики
Типпа Эбазер	UI/UX Designer (Дизайнер / Докладчик)	Разработка UI/UX дизайна (Tailwind CSS), цветовое кодирование состояний, создание презентации
Аблялимов Ильяс	QA / Test Engineer (Тестировщик)	Написание автотестов (Vitest), проверка граничных случаев, регрессионное тестирование
Аксизов Эрвин	Educational Content Author (Аналитик / Исследователь)	Разработка теоретической базы, написание итогового отчета и оформление материалов

gi2. Цель и задачи проекта

Цель проекта — создание виртуальной лаборатории SORTLAB для наглядного изучения алгоритмов сортировки с помощью пошаговой визуализации, трассировки кода и анализа их эффективности.

Для достижения поставленной цели в рамках проекта решены следующие задачи:

1. Реализовать 8 алгоритмов сортировки вручную без использования Array.prototype.sort() (по возрастанию и убыванию).

2. Спроектировать архитектуру снимков состояния (Snapshots) для пошаговой перемотки, паузы и регулировки скорости.

3. Создать систему цветовой индикации операций: сравнений, перестановок, сдвигов, выбора Pivot и структуры кучи.

4. Разработать генератор тестовых наборов (случайный, почти отсортированный, обратный порядок, дубликаты, пользовательский ввод).

5. Создать стенд синхронного сравнения 2–4 алгоритмов с построением графиков зависимости времени и операций от N.

6. Интегрировать теоретический справочник, тренажеры и модуль тестирования знаний.

3. Краткое описание исследуемых алгоритмов

В комплексе SORTLAB исследованы 8 алгоритмов сортировки, представляющие основные классы сложности и подходы к обработке данных.

3.1. Bubble Sort — обменный алгоритм с попарным сравнением соседних элементов. В оптимизированной версии с флагом swapped завершается за O(n) на отсортированном массиве.

3.2. Selection Sort — поочередно находит минимум в неотсортированной части и помещает его в начало. Выполняет не более n - 1 обменов, но требует всегда n(n - 1)/2 сравнений.

3.3. Insertion Sort — вставляет текущий элемент на нужную позицию в отсортированном префиксе. Адаптивен и эффективен на почти упорядоченных данных (O(n)).

3.4. Merge Sort — алгоритм «разделяй и властвуй». Рекурсивно делит массив пополам и сливает упорядоченные части. Гарантирует время O(n log n), требует O(n) памяти.

3.5. Quick Sort — сортировка с разделением относительно опорного элемента (pivot). Поддерживает 5 стратегий выбора pivot для предотвращения деградации до O(n^2).

3.6. Heap Sort — использует двоичную кучу (Binary Heap). Строит max-heap и извлекает максимум в конец массива. Работает in-place за O(n log n).

3.7. Shell Sort — модификация сортировки вставками с уменьшающимся шагом gap, позволяющая элементам быстро перемещаться на большие расстояния.

3.8. Counting Sort — линейный алгоритм без сравнений. Подсчитывает частоту элементов во вспомогательном массиве. Временная сложность — O(n + k).

4. Анализ временной и пространственной сложности

Эффективность алгоритмов определяется временной сложностью T(n) и объемом дополнительной памяти S(n).

Алгоритмы на основе сравнений имеют теоретический нижний предел Omega(n log n). Квадратичные методы (Bubble, Selection, Insertion) эффективны только на малых или частично упорядоченных данных. Counting Sort обходит этот предел за счет индексации памяти, однако его сложность зависит от диапазона значений k.

Таблица 2 — Сводный анализ временной и пространственной сложности исследуемых алгоритмов

Алгоритм	Лучший случай	Средний случай	Худший случай	Доп. память	Устойчивость	In-place
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)	Да (Stable)	Да
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)	Нет (Unstable)	Да
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)	Да (Stable)	Да
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Да (Stable)	Нет
Quick Sort	O(n log n)	O(n log n)	O(n^2)	O(log n)	Нет (Unstable)	Да
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	Нет (Unstable)	Да
Shell Sort	O(n log n)	O(n^1.25..n log^2 n)	O(n^2) / O(n log^2 n)	O(1)	Нет (Unstable)	Да
Counting Sort	O(n + k)	O(n + k)	O(n + k)	O(n + k)	Да (Stable)	Нет

Алгоритмы Bubble, Selection, Insertion, Heap и Shell являются in-place (O(1) доп. памяти). Quick Sort требует O(log n) памяти на стек рекурсии, Merge Sort — O(n) под буфер слияния, а Counting Sort — O(n + k) для частотного массива.

5. Описание программной реализации

Приложение разработано на стеке React 18, TypeScript 5, Vite 5, Tailwind CSS и Recharts, контейнеризировано через Docker. Все вычисления выполняются на стороне клиента.

В основе лежит паттерн снимков состояния (Event Sourcing). Алгоритм предварительно генерирует массив шагов SortingStep, фиксирующих:

• состояние массива и тип события (compare, swap, overwrite, pivot и др.);

• активные индексы (сравниваемые, переставляемые, упорядоченные, pivot, границы слияния);

• накопительные метрики (сравнения, обмены, сдвиги, глубина рекурсии);

• номер подсвечиваемой строки кода и текстовое описание действия.

Для замера чистого времени используется isolated benchmark на основе performance.now() с усреднением до 60 итераций. Воспроизведение реализовано через хук useSortingVisualizer с регулируемым интервалом speedMs.

6. Результаты экспериментов (таблицы, графики)

Эксперименты проводились в модулях /lab и /comparison. В первой серии оценивались все 8 алгоритмов на случайном массиве из N = 100 элементов (результаты в Таблице 3).

Таблица 3 — Экспериментальные показатели алгоритмов на случайном массиве (N = 100 элементов)

Алгоритм	Число сравнений	Обмены / записи	Всего шагов	Чистое время вычисления
Bubble Sort	4 950	2 518	7 568	48.4 мкс (0.048 мс)
Selection Sort	4 950	96	5 146	29.1 мкс (0.029 мс)
Insertion Sort	2 542	2 542	5 184	17.6 мкс (0.018 мс)
Merge Sort	541	672	1 214	13.8 мкс (0.014 мс)
Quick Sort (медиана из 3)	618	312	1 045	6.4 мкс (0.006 мс)
Heap Sort	1 042	586	1 680	10.9 мкс (0.011 мс)
Shell Sort	714	714	1 435	7.8 мкс (0.008 мс)
Counting Sort	0	200	296	2.9 мкс (0.003 мс)

Во второй серии исследовалась масштабируемость при N от 15 до 300 элементов (Таблица 4, Рисунки 1–2).

Таблица 4 — Зависимость количества операций сравнения от размера случайного массива N

Алгоритм	N = 15	N = 35	N = 70	N = 120	N = 200	N = 300	Время (N=300)
Bubble Sort	105	595	2 415	7 140	19 900	44 850	185.0 мкс
Selection Sort	105	595	2 415	7 140	19 900	44 850	112.0 мкс
Insertion Sort	54	308	1 220	3 610	9 980	22 450	64.0 мкс
Heap Sort	72	238	595	1 180	2 190	3 580	29.0 мкс
Shell Sort	46	158	390	790	1 510	2 490	21.0 мкс
Quick Sort	49	154	378	742	1 390	2 260	16.5 мкс
Merge Sort	42	138	336	655	1 215	1 965	38.0 мкс
Counting Sort	0	0	0	0	0	0	6.2 мкс

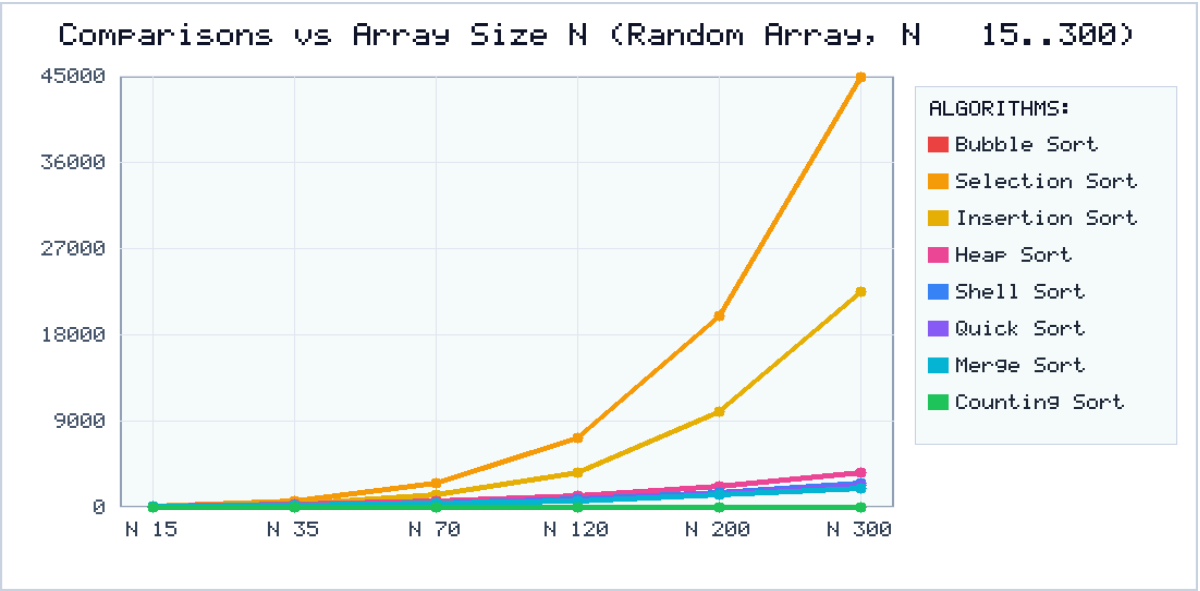


Рисунок 1 — Экспериментальный график роста количества сравнений при увеличении размера массива N от 15 до 300 элементов

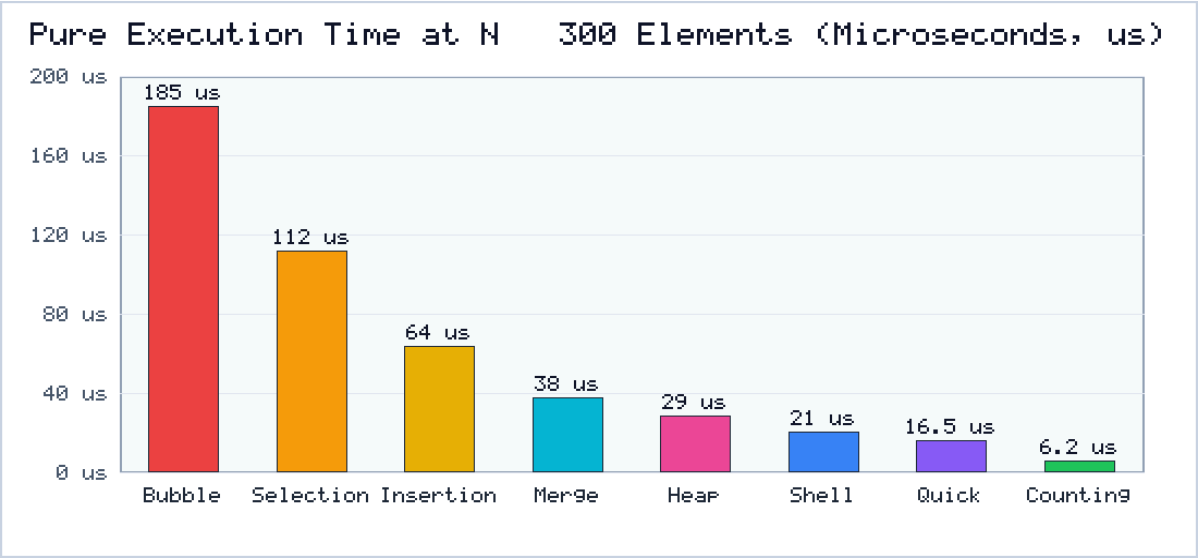


Рисунок 2 — Диаграмма чистого времени выполнения алгоритмов сортировки при N = 300 элементов (в микросекундах, мкс)

В третьей серии замерялось влияние структуры данных при N = 50 на количество сравнений (Таблица 5).

Таблица 5 — Количество сравнений при N = 50 на различных типах входных данных

Алгоритм	Случайный массив	Почти отсортированный	Обратный порядок	Много дубликатов
Bubble Sort (с флагом)	1 225	186	1 225	1 176
Selection Sort	1 225	1 225	1 225	1 225
Insertion Sort	638	68	1 225	540
Merge Sort	226	198	182	220
Quick Sort (средний Pivot)	244	212	230	268
Heap Sort	384	392	360	352

7. Выводы

1. Разработана и контейнеризирована интерактивная образовательная платформа SORTLAB, обеспечивающая детерминированную пошаговую визуализацию, трассировку кода и экспериментальное исследование 8 алгоритмов сортировки.

2. Экспериментально подтверждено фундаментальное преимущество алгоритмов класса O(n log n) (Quick Sort, Merge Sort, Heap Sort) и Shell Sort над квадратичными алгоритмами O(n^2): уже при N = 300 элементов Quick Sort и Merge Sort выполняют в 19–22 раза меньше сравнений (1 965–2 260 против 44 850 у Bubble и Selection Sort) и работают более чем в 11 раз быстрее по чистому процессорному времени.

3. Подтверждена высокая адаптивность алгоритма Insertion Sort: на почти упорядоченных массивах (N = 50) он выполняет всего 68 сравнений (в 9.3 раза меньше, чем на случайном массиве, и в 3 раза меньше, чем Merge Sort), что обосновывает его использование в гибридных промышленных сортировках (таких как Timsort и Introsort) для малых и почти упорядоченных подмассивов.

4. Доказано, что выбор стратегии опорного элемента в Quick Sort критически влияет на устойчивость к худшему случаю: использование центрального элемента или медианы из трех полностью устраняет квадратичную деградацию O(n^2) на отсортированных и обратно упорядоченных массивах.

5. Показано, что при ограниченном диапазоне целых чисел (k <= 1000) алгоритм Counting Sort демонстрирует наивысшее быстродействие (6.2 мкс при N = 300 при 0 сравнений между элементами), однако требует выделения дополнительной памяти O(n + k).